

Vector Rally - Relazione

Marta Musso

Settembre 2024

Contents

1	Introduzione	3
1.1	Regole del Gioco	3
2	Responsabilità Individuate	4
2.1	Model	4
2.2	View	6
2.3	Controller	7
3	Principi SOLID e Pattern Utilizzati	9
3.1	Principi SOLID	9
3.1.1	Single Responsibility Principle	9
3.1.2	Open/Closed Principle	9
3.1.3	Liskov Substitution Principle	9
3.1.4	Interface Segregation Principle	10
3.1.5	Dependency Inversion Principle	10
3.2	Pattern Utilizzati	10
3.2.1	Factory Pattern	10
3.2.2	Strategy Pattern	10
3.2.3	Singleton Pattern	10
4	Strumenti Utilizzati	11
4.1	Gradle	11
4.2	Javadoc	12
4.3	JUnit Test	13
4.4	JavaFX	14
4.5	Scene Builder	14
4.6	GitHub	14
4.7	Costrutti Visti a Lezione	15
5	Istruzioni per Eseguire il Progetto e Link Utili	16
5.1	Run e Build	16

5.2	Link Utili	16
6	Implementazione della Specifica del Progetto	17
6.1	Simulazione da Terminale	17
6.2	Simulazione tramite Interfaccia Grafica	18
6.3	Istruzioni per le Simulazioni	18
7	Conclusione e sviluppi futuri	19
7.1	Sviluppi Futuri	19

Chapter 1

Introduzione

Vector Rally è un gioco di simulazione di corse automobilistiche a turni, progettato per essere giocato sia tramite interfaccia a riga di comando (CLI) che con una semplice interfaccia grafica (GUI) utilizzando JavaFX. Il gioco consente ai giocatori di controllare le loro auto su una pista, facendo uso di algoritmi per determinare le mosse valide e gestendo le interazioni tra i giocatori e la pista.

Il gioco si svolge su tracciati rappresentati da file di testo (`.txt`), in cui ogni simbolo corrisponde a una specifica componente della pista. Ad esempio, i simboli possono rappresentare la linea di partenza e quella di arrivo.

1.1 Regole del Gioco

Il gioco segue le seguenti regole fondamentali:

- **Obiettivo:** Vince il primo giocatore che raggiunge la linea di arrivo.
- **Eliminazione:** Un giocatore senza mosse valide viene eliminato dalla partita.
- **Mosse:** Ad ogni turno, i giocatori possono muovere le loro auto in base alle mosse valide determinate dall'algoritmo. Le mosse valide sono determinate dalla posizione attuale dell'auto, dalla sua attuale accelerazione e dalla configurazione del tracciato.

Chapter 2

Responsabilità Individuate

Il progetto Vector Rally è stato sviluppato seguendo il pattern Model-View-Controller (MVC). Questo approccio facilita la separazione delle responsabilità, migliorando la manutenibilità e l'estensibilità del codice. Di seguito vengono dettagliate le responsabilità principali e le rispettive classi e interfacce che le implementano.

2.1 Model

Il modello contiene la logica del gioco e rappresenta i dati con cui l'applicazione lavora. Comprende le classi che definiscono i giocatori, le auto, i movimenti, le piste, gli algoritmi, le strategie e le regole del gioco.

- **Definire i Giocatori:** Gestire le informazioni e le azioni dei giocatori, inclusi i loro attributi e comportamenti.
 - `Player` - Interfaccia per i giocatori.
 - `DefaultPlayer` - Classe astratta per un giocatore di default.
 - `HumanPlayer` - Rappresenta un giocatore umano.
 - `BotPlayer` - Rappresenta un giocatore bot.
- **Gestire le Auto:** Rappresentare le auto dei giocatori, inclusi colori, stati e movimenti.
 - `Car` - Interfaccia per le auto.
 - `RaceCar` - Implementa un'auto da corsa.
 - `CarColour` - Enumera i colori delle auto.

- **Gestire i Movimenti:** Definire e validare i movimenti delle auto e le loro posizioni sulla pista.
 - `Coordinates` - Interfaccia per le coordinate di una posizione.
 - `Vector` - Interfaccia per un vettore di movimento.
 - `Position` - Rappresenta una posizione sulla pista.
 - `Acceleration` - Rappresenta l'accelerazione di un'auto.
 - `Move` - Record che rappresenta una mossa.
 - `Direction` - Enumera le direzioni di un movimento.
- **Rappresentare il Tracciato:** Definire i vari componenti della pista, come la linea di partenza, la linea di arrivo, le strade e gli ostacoli. Gestire la costruzione di un tracciato.
 - `Track` - Interfaccia per il tracciato.
 - `RaceTrack` - Rappresenta la pista da corsa.
 - `TrackComponent` - Enumera i componenti del tracciato.
 - `RaceTrackBuilder` - Costruisce il tracciato a partire da un file `.txt`.
- **Implementare le Regole del Gioco:** Stabilire le regole per il gioco, compresa la validazione delle mosse e l'interazione con i componenti del tracciato.
 - `MoveValidator` - Interfaccia per la validazione delle mosse.
 - `BasicMoveValidator` - Valida i movimenti base.
 - `BasicMovesGenerator` - Genera i movimenti base.
- **Implementare Algoritmi di Movimento:** Definire e implementare algoritmi per generare mosse basate sulle posizioni e i tracciati.
 - `AStar` - Implementa l'algoritmo A* per la ricerca del percorso.
 - `Node` - Rappresenta un nodo nell'algoritmo A*.
 - `NeighborsGenerator` - Interfaccia per generare i vicini di una posizione.
 - `FourNeighborsGenerator` - Implementa un generatore di vicini su quattro direzioni.
 - `EightNeighborsGenerator` - Implementa un generatore di vicini su otto direzioni.

- **Definire Strategie per i Bot:** Stabilire le strategie decisionali per i bot.
 - `DecisionStrategy` - Interfaccia per le strategie decisionali dei bot.
 - `EasyBotStrategy` - Implementa una strategia facile per i bot.
 - `MediumBotStrategy` - Implementa una strategia media per i bot.
 - `HardBotStrategy` - Implementa una strategia difficile per i bot.
 - `BotStrategy` - Enumera le strategie dei bot.

2.2 View

La vista è responsabile della presentazione dei dati all'utente. Include le classi che gestiscono l'input e l'output.

- **Gestione dell'Input/Output:** Raccogliere input dall'utente e visualizzare messaggi e dati di gioco.
 - `IOController` - Interfaccia per definire le principali azioni di I/O.
 - `TerminalIOController` - Implementazione dell'I/O per la CLI.
 - `GameMessageProvider` - Implementazione per la fornitura di messaggi di gioco.
- **Definizione di interazioni specifiche per ogni fase del gioco:** Gestire le azioni predefinite di ciascuna fase del gioco, dalle impostazioni iniziali fino al termine della partita, definendo le interazioni tra utente e software e il modo in cui mostrarle.
 - `SetupGameView` - Interfaccia per definire le principali azioni che riguardano la scelta e la presentazione di impostazioni iniziali della partita da parte dell'utente.
 - `CLISetupGameView` - Implementazione della precedente interfaccia per la CLI.
 - `MatchGameView` - Interfaccia per definire il comportamento e l'interazione tra utente e gioco nella partita vera e propria, oltre al modo in cui presentare l'avanzamento del match.
 - `CLIMatchGameView` - Implementazione della precedente interfaccia per la CLI.

- **FinishGameView** - Interfaccia per definire le interazioni da gestire al termine di una partita.
- **CLIFinishGameView** - Implementazione della precedente interfaccia per la CLI.

2.3 Controller

Il controller gestisce il flusso del gioco e delle interazioni dell'utente con il sistema. Include le classi che orchestrano le operazioni di setup e gestione del match.

- **Setup della Partita:** Gestire la creazione del setup della partita, inclusa la lista dei giocatori e il tracciato creato.
 - **GameSetup** - Interfaccia per il setup del gioco.
 - **BasicGameSetup** - Implementazione generica del setup del gioco (indipendente dal tipo di interfaccia utilizzata).
 - **SetupResult** - Record risultante dal setup del gioco.
- **Gestione e Arbitraggio del Match:** Controllare e gestire l'avvio del gioco, il flusso della partita comprendente le condizioni e le conseguenze per le eliminazioni e le vittorie e la gestione esterna dei turni.
 - **MatchController** - Interfaccia per il controllo del match.
 - **BasicMatchController** - Implementazione generica del controllo del match.
 - **GameEngine** - Interfaccia per il controllo del motore di gioco.
 - **BasicGameEngine** - Implementazione generica della precedente interfaccia.
 - **TurnHandler** - Interfaccia per il controllo della coda dei turni.
 - **BasicTurnHandler** - Implementazione generica del controllo dei turni.
- **Costruzione e controllo delle componenti necessarie:** Controllare e gestire la corretta costruzione e validazione del tracciato, delle strategie dei bot e della lista di percorsi contenenti i files che rappresentano le piste.
 - **BotStrategyFactory** - Factory per assegnare le strategie ai bot.

- **TrackBuilder** - Interfaccia per definire la costruzione di un tracciato.
- **RaceTrackBuilder** - Implementazione della precedente interfaccia con l'utilizzo specifico delle componenti definite nel progetto.
- **TrackPathBuilder** - Definisce metodi per trovare i percorsi dei files disponibili per essere utilizzati come piste.

Chapter 3

Principi SOLID e Pattern Utilizzati

In questo capitolo vengono analizzati i principi SOLID seguiti durante lo sviluppo del progetto Vector Rally, insieme ai pattern di progettazione utilizzati.

3.1 Principi SOLID

I principi SOLID sono linee guida che aiutano a mantenere il codice flessibile e manutenibile.

3.1.1 Single Responsibility Principle

Ogni classe dovrebbe avere una sola responsabilità.

- `BasicMoveValidator` ha la sola responsabilità di validare i movimenti.

3.1.2 Open/Closed Principle

Le classi dovrebbero essere aperte per l'estensione ma chiuse per la modifica.

- `RaceTrack` può essere estesa per aggiungere nuovi tipi di componenti del tracciato senza alterare la struttura base.

3.1.3 Liskov Substitution Principle

Le classi derivate devono poter essere sostituite alle classi base senza alterare la correttezza del programma.

- L'interfaccia `Player` e le sue implementazioni possono essere utilizzate in modo intercambiabile nel metodo `handleTurn` di `BasicMatchController` senza alterare il comportamento del gioco.

3.1.4 Interface Segregation Principle

Le interfacce dovrebbero essere specifiche per il client, evitando di creare interfacce grandi e generiche.

- `GameEngine`, `MatchController` e `TurnHandler` sono interfacce separate per gestire una partita.

3.1.5 Dependency Inversion Principle

Le dipendenze dovrebbero essere invertite rispetto alle astrazioni.

- `BasicMatchController` dipende da `MatchController` per la configurazione del gioco, consentendo la sostituzione delle implementazioni senza modificare il motore del gioco.

3.2 Pattern Utilizzati

Nel progetto Vector Rally sono stati utilizzati alcuni pattern di progettazione per migliorare la struttura del codice, rendendolo più flessibile ed estensibile.

3.2.1 Factory Pattern

Il Factory Pattern è utilizzato per creare oggetti senza esporre la logica di creazione al client. Questo pattern è impiegato nella classe `BotStrategyFactory` per creare le strategie dei bot.

3.2.2 Strategy Pattern

Il Strategy Pattern è utilizzato per definire una famiglia di algoritmi, incapsularli e renderli intercambiabili. Questo pattern è utilizzato per determinare le strategie decisionali dei bot.

3.2.3 Singleton Pattern

Il Singleton Pattern è utilizzato per garantire che esista una sola istanza di `SceneManager` durante l'esecuzione dell'applicazione..

Chapter 4

Strumenti Utilizzati

In questo capitolo vengono descritti gli strumenti utilizzati durante lo sviluppo del progetto Vector Rally. Ogni sezione fornisce una panoramica dello strumento e degli esempi di codice per illustrarne l'uso.

4.1 Gradle

Gradle è stato utilizzato per gestire le dipendenze e automatizzare il processo di build del progetto. Grazie a Gradle, è stato possibile configurare facilmente le librerie necessarie e gestire le task di compilazione, esecuzione e test.

Esempio di file `build.gradle`:

```
plugins {  
    id 'application'  
    id 'org.openjfx.javafxplugin' version '0.1.0'  
    id("buildlogic.java-application-conventions")  
}  
  
repositories {  
    mavenCentral()  
}  
  
run {  
    standardInput = System.in  
}  
  
dependencies {  
    implementation 'org.apache.commons:commons-text:1.10.0'}
```

```

        implementation 'org.openjfx:javafx-controls:18.0.1'
        implementation 'org.openjfx:javafx-fxml:18.0.1'
        implementation project(':api')
    }

    java {
        toolchain {
            languageVersion = JavaLanguageVersion.of(21)
        }
    }

    javafx {
        version = "22.0.1"
        modules = [ 'javafx.controls', 'javafx.fxml' ]
    }

    application {
        mainClass = 'it.unicam.cs.mpmgc.vetorally.app.GraphicalApp'
    }

```

4.2 Javadoc

Javadoc è stato utilizzato per documentare il codice. Generando la documentazione HTML direttamente dal codice sorgente, è possibile mantenere la documentazione sempre aggiornata con l'evoluzione del codice.

Esempio di commento Javadoc:

```

/**
 * Retrieves the component at the specified position on the track.
 *
 * @param x the x coordinate.
 * @param y the y coordinate.
 * @return the {@link TrackComponent} at the specified position.
 */
TrackComponent getComponentAt(int x, int y);

```

4.3 JUnit Test

JUnit è stato utilizzato per testare il codice. Grazie a JUnit, è possibile scrivere test automatizzati per verificare la correttezza delle funzionalità implementate e garantire che le modifiche al codice non introducano bug.

Esempio di test JUnit:

```
public class BotPlayerTest {

    private BotPlayer botPlayer;
    private Car car;
    private BotStrategy strategy;

    @BeforeEach
    void setUp() {
        car = new RaceCar(CarColour.BLUE);
        strategy = BotStrategy.EASY;
        botPlayer = new BotPlayer(car, strategy);
    }

    @Test
    void constructorShouldThrowExceptionWhenCarIsNull() {
        assertThrows(NullPointerException.class,
            () -> new BotPlayer(null, strategy));
    }

    @Test
    void constructorShouldThrowExceptionWhenStrategyIsNull() {
        assertThrows(NullPointerException.class,
            () -> new BotPlayer(car, null));
    }

    @Test
    void getStrategyShouldReturnStrategy() {
        assertEquals(strategy, botPlayer.getStrategy());
    }
}
```

4.4 JavaFX

JavaFX è stato utilizzato per creare l'interfaccia grafica del gioco. Fornisce una ricca libreria di componenti UI e strumenti per costruire interfacce utente moderne e interattive.

Esempio di utilizzo di JavaFX:

```
@FXML
public void initialize() {
    nextTurnButton.setOnAction(event -> goToNextTurn());
    turnLabel.setText(messageProvider.getPreparationMessage());
}
```

4.5 Scene Builder

Scene Builder è stato utilizzato per progettare le interfacce utente in JavaFX. Permette di costruire le scene in modo visuale e di generare il file `FXML` corrispondente, facilitando la separazione tra logica e presentazione.

Esempio di codice `FXML` generato con Scene Builder:

```
<rowConstraints>
    <RowConstraints minHeight="10.0" prefHeight="30.0" />
    <RowConstraints minHeight="10.0" prefHeight="30.0" />
    <RowConstraints minHeight="10.0" prefHeight="30.0" />
</rowConstraints>
```

4.6 GitHub

GitHub è stato utilizzato per il controllo della versione e dello sviluppo del progetto nei suoi vari livelli di implementazione. Grazie a GitHub, è possibile tenere traccia delle modifiche e gestire i rami di sviluppo.

In particolare, sono stati utilizzati 3 rami differenti per gestire lo sviluppo del progetto nelle sue fasi differenti:

- `main`
- `development_base`
- `development_GUI`

4.7 Costrutti Visti a Lezione

Durante lo sviluppo del progetto, sono stati utilizzati diversi costrutti e concetti visti a lezione, come l'uso delle interfacce e dei record.

Esempio di record:

```
public record SetupResult(  
    NeighborsGenerator generator ,  
    RaceTrack raceTrack ,  
    List<Player> players) {  
}
```

Esempio di Generics:

```
public class BasicMovesGenerator<T extends NeighborsGenerator> {  
    private final T neighborsGenerator ;  
    private final MoveValidator moveValidator ;  
}
```

Chapter 5

Istruzioni per Eseguire il Progetto e Link Utili

5.1 Run e Build

Per costruire ed eseguire il progetto, utilizzare i seguenti comandi:

```
gradle build
gradle run
```

Verrà mostrata l'esecuzione dell'interfaccia grafica. Per la simulazione da terminale, eseguire **TerminalApp** dal modulo **app** o modificare il file **build.gradle** del modulo **app** come segue:

```
application {
    mainClass = 'it.unicam.cs.mpmgc.vectorrally.app.GraphicalApp'
}

application {
    mainClass = 'it.unicam.cs.mpmgc.vectorrally.app.TerminalApp'
}
```

5.2 Link Utili

Viene reso disponibile il riferimento al [Github Repository](#) contenente il codice del progetto Vector Rally.

Chapter 6

Implementazione della Specifica del Progetto

Il progetto Vector Rally consente di eseguire la simulazione di gioco sia da terminale sia tramite interfaccia grafica, come descritto nel Capitolo 5.

6.1 Simulazione da Terminale

La simulazione da terminale è stata curata in molti aspetti, prevedendo anche l'interazione di un utente umano. La simulazione da terminale è stata realizzata per essere abbastanza completa e sufficientemente dettagliata, occupando la maggior parte del tempo di realizzazione del progetto.

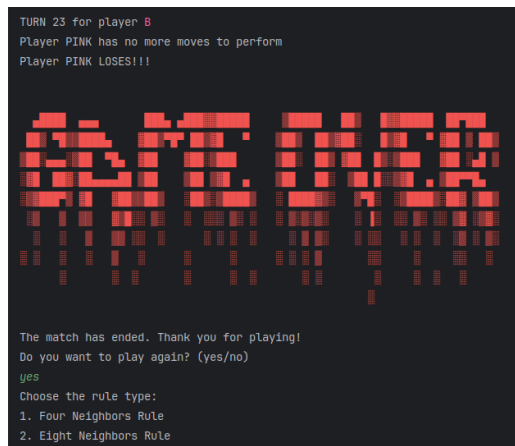


Figure 6.1: Sconfitta durante un match sul terminale

6.2 Simulazione tramite Interfaccia Grafica

Anche la simulazione tramite interfaccia grafica offre un sufficiente grado di interazione, permettendo all'utente di selezionare la mossa direttamente sul tracciato.

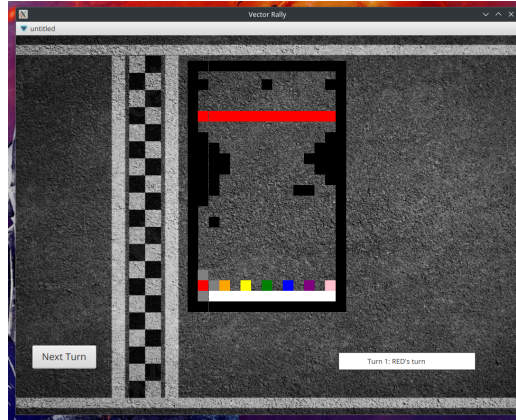


Figure 6.2: Setup iniziale del match su interfaccia grafica

6.3 Istruzioni per le Simulazioni

Per entrambe le simulazioni, è sufficiente seguire le istruzioni che vengono mostrate. La simulazione da terminale fornirà i prompt necessari per interagire con il gioco. Per quanto riguarda la simulazione da interfaccia grafica, è necessario avviare il gioco, configurare le caratteristiche della partita e iniziare la simulazione di gioco. Per selezionare la mossa desiderata interattivamente è sufficiente cliccare su uno dei rettangoli grigi che rappresentano le destinazioni disponibili.

Chapter 7

Conclusione e sviluppi futuri

Il codice è stato progettato per essere estensibile e modulare, facilitando l'aggiunta di nuove funzionalità e miglioramenti futuri.

7.1 Sviluppi Futuri

Il progetto Vector Rally è stato sviluppato con l'obiettivo di creare un codice modulare e aperto alle estensioni. Questo approccio permette di aggiungere nuove funzionalità senza dover modificare significativamente il codice esistente, garantendo una maggiore flessibilità e manutenibilità. Tra gli sviluppi futuri previsti ci sono:

- **Perfezionamento degli algoritmi di AI dei bot:** Testare ulteriori algoritmi che permettono la ricerca di cammini minimi per rendere più interessante la competizione.
- **Interfaccia Grafica Avanzata:** Rendere l'interfaccia grafica più avanzata e gradevole, includendo funzionalità quali il caricamento interattivo di nuovi tracciati e feedback visivi più intuitivi.